Network

Lecture 6

ETF2480 & ETF5248 - S2, 2026
Jessica Leung, PhD, CAP-X, SFHEA

Lecture's Goal

- Introduction
- Minimum Cost Network Flow
- Total Unimodularity
- Special Cases



Reading:

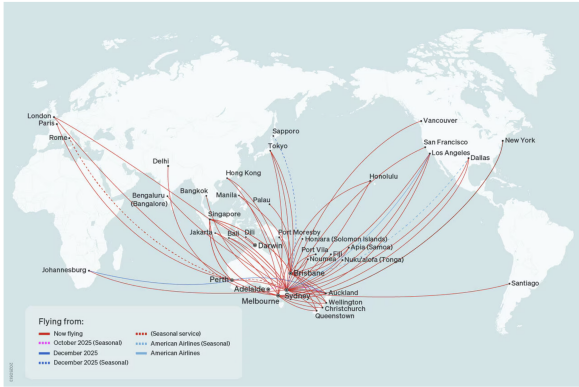
Bertsimas and Tsitsiklis, *Introduction to Linear Optimization*, Chapter 7.

Winston, *Operations Research*, Chapters 7 and 8.

Vanderbei, *Linear Programming*, Chapters 14 and 15.

Barabási, *Linked: The New Science of Networks*, Chapter 2.

Example of networks: Qantas International routes (2025)



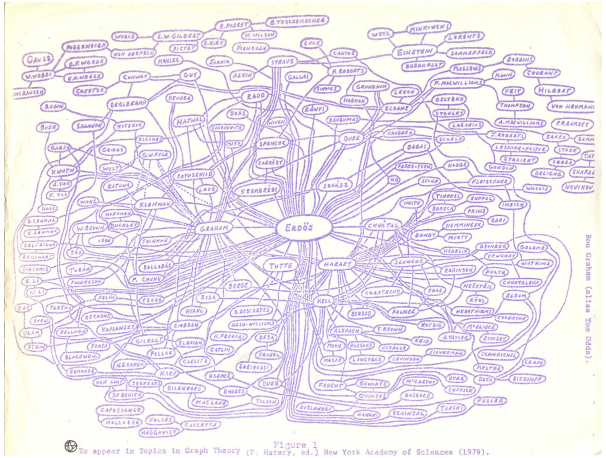
Source: https://www.qantas.com/au/en/qantas-experience/international-flight-network.html

Example of networks: Delta USA routes (2015)



Source: http://www.delta.com

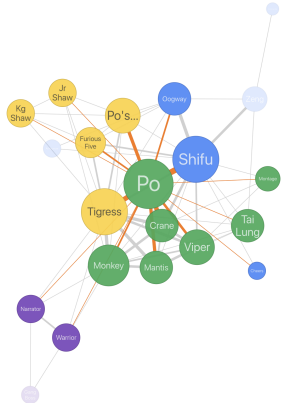
Erdös collaboration network



Source: http://www.math.cmu.edu/∼ctsourak/amazing.html

A related concept : The Bacon number

Social networks: Kung Fu Panda (2008)



Source: http://moviegalaxies.com/

Another interactive network:

https://graphics.straitstimes.com/STI/STIMEDIA/Interactives/2018/04/marvel-cinematic-universe-whos-who-interactive/index.html

Other examples

- Phone call networks
- World Wide Web
- Facebook
- Power grids
- ...

"*Networks are present everywhere. All we need is an eye for them.*"
A.-L. Barabási, **Linked: The New Science of Networks**

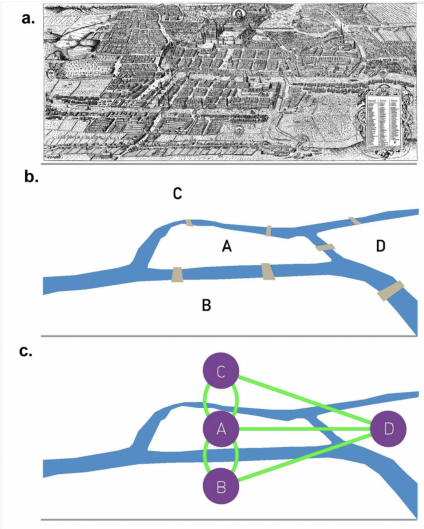
Link to Network Science Ebook

The origins of graph theory: Eulerian tour

- Few research fields can trace their birth to a single moment and place in history. The roots of Graph theory go back to 1735 in Königsberg, the capital of Eastern Prussia, a thriving merchant city of its time.
- Seven bridges were built across the river Pregel that surrounded the town. Five of these connected to the mainland the elegant island Kneiphof, caught between the two branches of the Pregel. The remaining two crossed the two branches of the river (see image on the next slides).
- This peculiar arrangement gave birth to a contemporary puzzle: **Can one walk across all seven bridges and never cross the same one twice?**
- Despite many attempts, no one could find such path. The problem remained unsolved until 1735, when Leonard Euler, a Swiss born mathematician, offered a rigorous mathematical proof that such path does not exist.
- Euler represented each of the four land areas separated by the river with letters A, B, C, and D. Next he connected with lines each piece of land that had a bridge between them. He thus built a graph, whose nodes were pieces of land and links were the bridges.
- Then Euler made a simple observation: if there is a path crossing all bridges, but never the same bridge twice, then nodes with odd number of links must be either the starting or the end point of this path. Indeed, if you arrive to a node with an odd number of links, you may find yourself having no unused link for you to leave it.

There is a video on Moodle showing an animation of the Königsberg bridges problem.

The origins of graph theory: Eulerian tour



- Euler's proof was the first time someone solved a mathematical problem using a graph.
- We learn that some problems become simpler and more tractable if they are represented as a graph.
- We also learn that the existence of the path does not depend on our ingenuity to find it. Rather, it is a property of the graph. Indeed, given the structure of the Königsberg graph, no matter how smart we are, we will never find the desired path. In other words, networks have properties encoded in their structure that limit or enhance their behavior.
- To understand the many ways networks can affect the properties of a system, we need to become familiar with graph theory, a branch of mathematics that grew out of Euler's proof.

Networks and Graphs

Network (directed graph, digraph): m nodes connected by n directed arcs

- arcs are ordered pairs of nodes: (u, v)
- we assume there is at most one arc from node u to node v
- there are no loops (no arcs (u, u))

Node-arc incidence matrix: $m \times n$ matrix A with entries

$$A_{ij} = \begin{cases} 1 & \text{if arc } j \text{ starts at node } i \\ -1 & \text{if arc } j \text{ ends at node } i \\ 0 & \text{otherwise} \end{cases}$$

Example

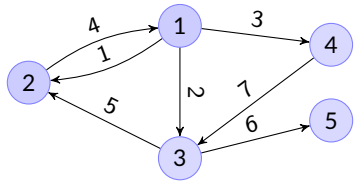


Figure: A network with 5 nodes and 7 arcs. Numbers on arcs show arc labels.

$$A = \begin{bmatrix} 1 & 1 & 1 & -1 & 0 & 0 & 0 \\ -1 & 0 & 0 & 1 & -1 & 0 & 0 \\ 0 & -1 & 0 & 0 & 1 & 1 & -1 \\ 0 & 0 & -1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & -1 & 0 \end{bmatrix}$$

Degree

- A key property of each node is its *degree*, representing the number of links it has to other nodes. The degree can represent the number of mobile phone contacts an individual has in the call graph (i.e. the number of different individuals the person has talked to), or the number of citations a research paper gets in the citation network.
- We denote with k_i the degree of the ith node in the network. For example, the degree of node C in the figure of the Eulerian tour is 3.
- In an undirected network the total number of links, L, can be expressed as the sum of the node degrees:

$$L = \frac{1}{2}\sum_{i=\mathbf{1}}^{N} k_i$$

  Here the 1/2 factor corrects for the fact that in the sum each link is counted twice.
- In a directed graph, we distinguish between incoming degree, k_i^{in}, representing the number of links that point **to** node i, and outgoing degree, k_i^{out}, representing the number of links that point **from** node i to other nodes. Finally, a node's total degree, k_i, is given by

$$k_i = k_i^{in} + k_i^{out}$$

- Once we have information for each node in a graph, we can compute summary statistics on the entire graph such as average degree, standard deviation of degree.

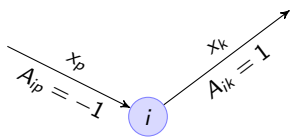
Network flow (Decision variable)

flow vector $x \in \mathrm{R}^n$

- x_j: flow (of material, traffic, charge, information, ...) through arc j
- positive if in direction of arc; negative otherwise

total flow leaving node i:

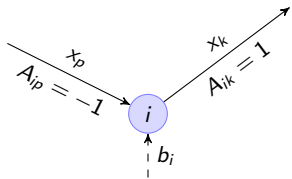
$$\sum_{j=1}^{n} A_{ij} x_j = (Ax)_i$$



External supply

Supply vector $b \in \mathrm{R}^m$

- b_i is external supply at node i (negative b_i represents external demand)



Balance equations:

$$Ax = b$$

Minimum cost network flow problem (MCNFP)

$$\begin{array}{rl} \text{minimize} & c^T x \\ \text{subject to} & Ax = b \\ & l \leq x \leq u \end{array}$$

- c_i is unit cost of flow through arc i
- l_j and u_j are limits on flow through arc j (typically, $l_j \leq 0$, $u_j \geq 0$)
- we assume $l_j < u_j$, but allow $l_j = -\infty$ and $u_j = +\infty$ to simplify notation

includes many network optimization problems as special cases

Totally unimodular matrix

Definition:

- a matrix is **totally unimodular** if all its minors are -1, 0, or 1
  (a minor is the determinant of a square submatrix)

Examples:

the matrix

$$A = \begin{bmatrix} 1 & 0 & -1 & 0 & 1 \\ 0 & -1 & 1 & -1 & -1 \\ 0 & 0 & 0 & 1 & 1 \end{bmatrix}$$

node-arc incidence matrix of a directed graph

Properties of a totally unimodular matrix A:

- the entries A_{ij} (i.e., its minors of order 1) are -1, 0, or 1
- the inverse of any nonsingular square submatrix of A has entries ± 1, 0

Integrality of extreme points

Let $\mathcal{P}$ be a polyhedron in $\mathbb{R}^n$ defined by

$$Ax = b, \qquad l \leq x \leq u$$

where

- A is totally unimodular
- b is an integer vector
- If there are finite bounds, then the finite lower bounds l and finite upper bounds u are integers.
- Integrality may break when we have finite bounds but non-integer.

then all the extreme points (vertices) of $\mathcal{P}$ are integer vectors.
This is also known as the *integrality theorem*.

Intuition: Why does this happen?

- What is an Extreme Point?
  In linear programming, the feasible region is a polyhedron (intersection of half-spaces). An extreme point is a“corner" of that polyhedron. The simplex method, for example, always moves from one extreme point to another, so extreme points are where optimal solutions are found.
- In general LPs, extreme points can be fractional. But in network flow LPs, something remarkable happens: If supplies/demands/capacities are integers, then every extreme point corresponds to integer flows.

Intuition: Why does this happen?

- For this to happen, two ingredients are needed:
  - Constraint structure is a Totally Unimodular (TU) matrix.
    The node–arc incidence matrix of a directed network is TU.
  - Integral RHS
    Supplies/demands (right-hand sides of balance equations) are integers. Capacities (if finite) are integers.

Why This is Powerful?

Consider a combinatorial optimization problem:

$$\begin{aligned} \text{minimize} \quad & c^T x \\ \text{subject to} \quad & Ax = b \\ & l \leq x \leq u \\ & x \in \mathrm{Z}^n \end{aligned}$$

This is an integer linear program, very difficult in general. The major workhorse for solving interger linear programs is the branch-and-bound algorithm, which is a smart way of doing exhaustive search in the solution set.

Why This is Powerful?

With integrality of extreme points, this problem is equivalent to this linear programming relaxation

$$\begin{aligned} \text{minimize} \quad & c^T x \\ \text{subject to} \quad & Ax = b \\ & l \leq x \leq u \end{aligned}$$

Implications for combinatorial optimization: Extreme optimal solution of the relaxation is also optimal for the IP. In other words, the optimal solution found at an extreme point is automatically integer-valued.
All you need is constraint matrix being TU matrix and RHS (and capacity if finite) integrality.

Special cases of the MCNFP

- Transportation problem
- Assignment problem
- Shortest path problem
- Maximum flow problem

Transportation problem

When network flow problem represents the shipment of goods along a transportation network, it is called the **transshipment problem**. An important special case is when the set of nodes N can be partitioned into two sets S and D,

$$N = S \cup D, \quad S \cap D = \varnothing,$$

such that every arc has its tail in S and its head in D.
The nodes in S are called **source** (or **supply**) nodes, while those in D are called **destination** (or **demand**) nodes.

Such graphs are called **bipartite graphs** (see next slide). A network flow problem on such a bipartite graph is called a **transportation problem**.

Bipartite graph

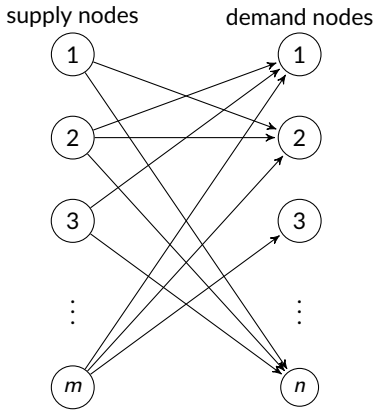


Figure: A bipartite graph - the network for a transportation problem.

Hitchcock transportation problem

When every supply node is connected to every demand node, the problem is called the **Hitchcock Transportation Problem**.

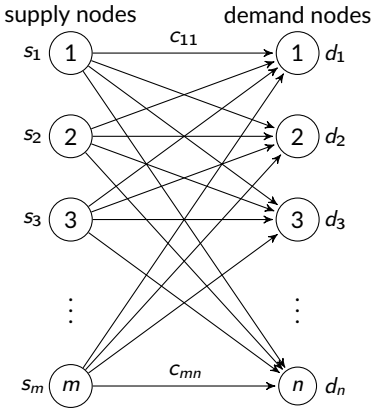


Figure: Every supply node is connected to every demand node.

Hitchcock transportation problem formulation

Let x_{ij} be the number of units shipped from supply point i to demand point j, then the problem can be formulated as:

$$
\begin{aligned}
\text{minimize} \quad & \sum_{i \in S} \sum_{j \in D} c_{ij} x_{ij} \\
\text{subject to} \quad & \sum_{j \in D} x_{ij} = s_i, \quad i \in S, \\
& \sum_{i \in S} x_{ij} = d_j, \quad j \in D, \\
& x_{ij} \geq 0, \quad i \in S,\ j \in D.
\end{aligned}
$$

Hitchcock transportation problem formulation

In **matrix notation**:

$$\begin{aligned} \text{minimize} \quad & c^T x \\ \text{subject to} \quad & Ax = b \\ & x \geq 0 \end{aligned}$$

where A is an $(m + n) \times mn$ node-arc incidence matrix, $b = (s, -d)$, $l = 0$, $u = +\infty$

Example: transportation problem

Powerco has three electric power plants that supply the needs of four cities. Each power plant can supply the following numbers of kilowatts-hours (kWh) of electricity: plant 1 - 35 million; plant 2 - 50 million; plant 3 - 40 million.

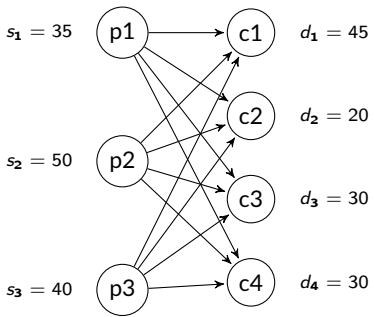
The peak power demands in these cities, which occur at the same time (2 pm), are as follows (in kWh): city 1 - 45 million; city 2 - 20 million; city 3 - 30 million; city 4 - 30 million.

The cost of sending 1 million kWh of electricity from plant to city depends on the distance the electricity must travel. Formulate an LP to minimise the cost of meeting each city's peak power demand.

Table: shipping cost, supply and demand.

	to				
from	city 1	city 2	city 3	city 4	**supply**
plant 1	$8	$6	$10	$9	35
plant 2	$9	$12	$13	$7	50
plant 3	$14	$9	$16	$5	40
demand	45	20	30	30	

Example: transportation problem



Let x_{ij} be the flow from the ith supply node to the jth demand node (in millions of kWh). Then the problem can be formulated as

$$\text{minimize} \quad \sum_{i=1}^{3} \sum_{j=1}^{4} c_{ij} x_{ij}$$

$$\text{subject to} \quad \sum_{j=1}^{4} x_{ij} = s_i, \quad \forall i; \quad \sum_{i=1}^{3} x_{ij} = d_j, \quad \forall j;$$

$$x_{ij} \geq 0 \quad \forall i, \forall j.$$

Live Coding Time!

Let’s code the unbalanced transportation problem together!

Balancing a transportation problem

The problem is said to be a **balanced** transportation problem if

$$\sum_{i \in S} s_i = \sum_{j \in D} d_j,$$

otherwise the problem is **unbalanced**. if total supply exceeds total demand ($\sum_{i \in S} s_i > \sum_{j \in D} d_j$):

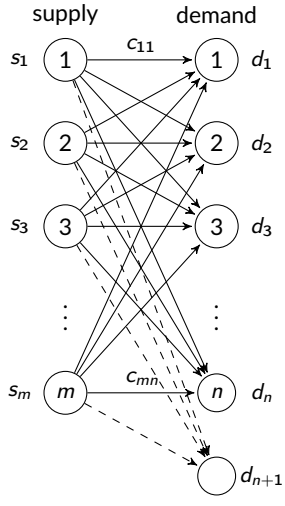
- we can balance the problem by adding a **dummy demand node**
- since shipments to the dummy demand node are not real, they are assigned a cost of zero

if total supply is strictly less than total demand ($\sum_{i \in S} s_i < \sum_{j \in D} d_j$):

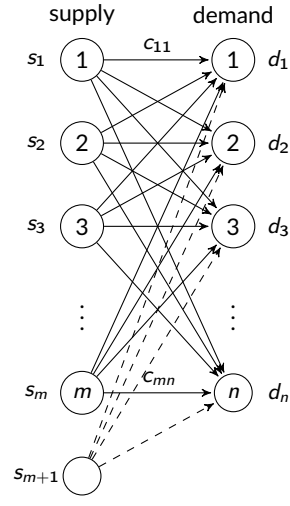
- the problem is infeasible
- however, it is sometimes desirable to allow the possibility of leaving some demand unmet
- we can balance the problem by adding a **dummy supply node** and associate a penalty with unmet demand

Balancing a transportation problem

supply exceeds demand



demand exceeds supply



Assignment problem

- there are m **persons** (set S) and m **projects** (set D)
- c_{ij} is the cost of person i completing project j
- we wish to assign a different person to each project to minimize total cost
- $x_{ij} = 1$ if person i is assigned to project j, and $x_{ij} = 0$ otherwise

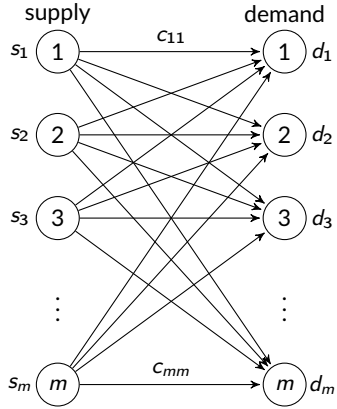
combinatorial optimization problem, very difficult in general

$$
\begin{aligned}
\text{minimize} \quad & \sum_{i \in S} \sum_{j \in D} c_{ij} x_{ij} \\
\text{subject to} \quad & \sum_{j \in D} x_{ij} = 1 \qquad i \in S, \\
& \sum_{i \in S} x_{ij} = 1 \qquad j \in D, \\
& x_{ij} \in \{0, 1\} \qquad i \in S,\ j \in D.
\end{aligned}
$$

Network for an assignment problem

Assignment problem is a special case of transportation problem:

- the number of supply nodes is equal to the number of demand nodes
- each supplier has unit supply and each consumer has unit demand



Assignment as a transportation problem

In **matrix notation**:

$$\begin{aligned} \text{minimize} \quad & c^T x \\ \text{subject to} \quad & Ax = b \\ & x \in \{0,1\}^{m^2} \end{aligned}$$

where A is an $2m \times m^2$ node-arc incidence matrix, $b = (1, -1)$

- equivalent to its linear programming relaxation:

$$\begin{aligned} \text{minimize} \quad & c^T x \\ \text{subject to} \quad & Ax = b \\ & 0 \leq x \leq 1 \end{aligned}$$

- because A is totally unimodular and b, l and u are integer vectors

Example: assignment problem

Machineco has four machines and four jobs to be completed. Each machine must be assigned to complete one job. The time required to setup each machine for completing each job is shown in the table. Machineco wants to minimize the total setup time needed to complete the four jobs.

Table: setup cost

	time (hours)			
	job 1	job 2	job 3	job 4
machine 1	14	5	8	7
machine 2	2	12	6	5
machine 3	7	8	3	9
machine 4	2	4	6	10

Shortest path problem

- Suppose each arc in the network has a length associated with it
- We want to find a **shortest path** from node 1 to node m in the network
- Common applications: vehicle and internet routing
- **example**: find the shortest path between nodes 1 and 7 in the following network:

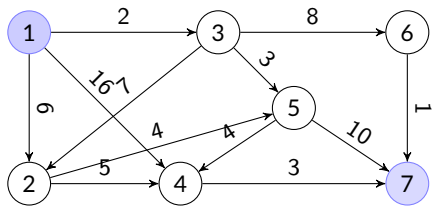


Figure: Network for a shortest path problem. Labels on arcs indicate length.

How to formulate this problem mathematically?

Shortest path problem: formulation

shortest path in directed graph with node-arc incidence matrix A
(Forward) paths from node 1 to m can be represented by vectors x with

$$Ax = (1, 0, \ldots, 0, -1), \qquad x \in \{0, 1\}^n$$

shortest path is solution of

$$\begin{array}{rl} \text{minimize} & c^T x \\ \text{subject to} & Ax = (1, 0, \ldots, 0, -1); x \in \{0, 1\}^n \end{array}$$

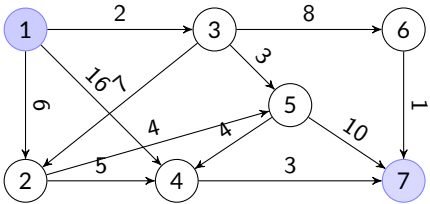
LP formulation

$$\begin{array}{rl} \text{minimize} & c^T x \\ \text{subject to} & Ax = (1, 0, \ldots, 0, -1); \quad 0 \leq x \leq 1 \end{array}$$

optimal solutions satisfy $x_i \in \{0, 1\}$

Example

To find the shortest path between nodes 1 and 7 in the following network:



with node-arc incidence matrix

$$
\begin{array}{ll}
\text{minimize} & c^T x \\
\text{subject to} & Ax = (1, 0, \ldots, 0, -1) \\
& 0 \leq x \leq 1
\end{array}
$$

$$
A = \begin{bmatrix}
1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
-1 & 0 & 0 & 1 & 1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & -1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\
0 & 0 & -1 & -1 & 0 & 0 & 0 & 0 & 1 & -1 & 0 & 0 \\
0 & 0 & 0 & 0 & -1 & 0 & -1 & 0 & 0 & 1 & 1 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & -1 & -1
\end{bmatrix}
$$

Non-obvious example: shortest path problem

Suppose it costs $10,000 to purchase a new car. The annual operating cost and resale value of a used car are shown in the following table. Assuming that one now has a new car, determine a replacement policy that minimizes the net costs of owning and operating a car for the next six years.

Age of car (Years)	Resale Value (S)	Operating Cost ($)	
1	7,000	300	(year 1)
2	6,000	500	(year 2)
3	4,000	800	(year 3)
4	3,000	1,200	(year 4)
5	2,000	1,600	(year 5)
6	1,000	2,200	(year 6)

Non-obvious example: shortest path problem

Let’s draw the network together.

Non-obvious example: shortest path problem

Formulation:

Numerical Solution: $p^* = 14400$,
$x^* = (0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0)$
Conclusion and Interpretation: Replace the car at times 2, 4, and 6. Total cost is minimised at \$14,400.

Maximum flow problem

- suppose each arc in the network has a capacity associated with it
- maximize flow from node 1 (source) to node m (sink) through the network

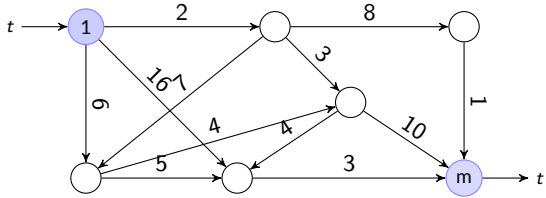


Figure: Network for a maximum flow problem. Labels on arcs indicate capacity.

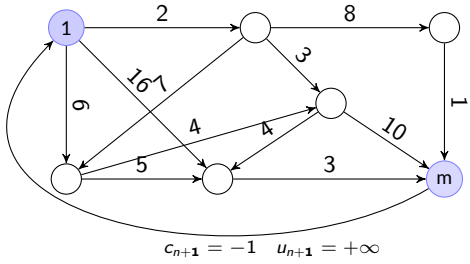
$$\begin{array}{ll} \text{maximize} & t \\ \text{subject to} & Ax = tb \\ & l \leq x \leq u \end{array}$$

where A is node-arc incidence matrix and $b = (1, 0, \ldots, 0, -1)$

Maximum flow reformulated as a MCNFP

maximum flow problem can be reformulated as a minimum cost network flow problem

- let the cost of every arc $c_{ij} = 0$;
- introduce a new arc $(m, 1)$ with $u_{m,\mathbf{1}} = +\infty$ and $c_{m,\mathbf{1}} = -1$.



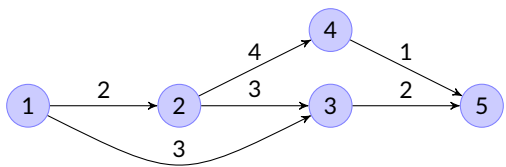
$$\begin{aligned} \text{minimize} \quad & -t \\ \text{subject to} \quad & \begin{bmatrix} A & -b \end{bmatrix} \begin{bmatrix} x \\ t \end{bmatrix} = 0; \quad l \leq x \leq u \end{aligned}$$

Note: matrix $[A \ -b]$ is node-arc incidence matrix, hence totally unimodular

Example: maximum flow problem

Sunco Oil wants to ship the maximum possible amount of oil (per hour) via pipeline from node 1 to node 5. On its way from node 1 to node 5, oil must pass through some or all of stations 2, 3, and 4. The various arcs represent pipelines of different diameters. The maximum number of barrels of oil that can be pumped through each arc is shown above the arcs.

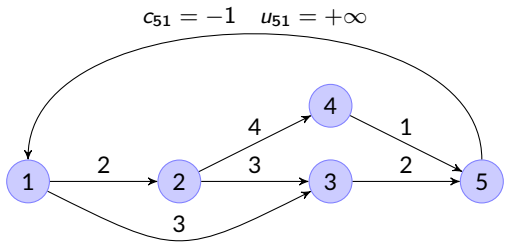
Formulate an LP that can be used to determine the maximum number of barrels of oil per hour that can be sent from 1 to 5.



Example: maximum flow problem

Sunco Oil wants to ship the maximum possible amount of oil (per hour) via pipeline from node 1 to node 5. On its way from node 1 to node 5, oil must pass through some or all of stations 2, 3, and 4. The various arcs represent pipelines of different diameters. The maximum number of barrels of oil that can be pumped through each arc is shown above the arcs.

Formulate an LP that can be used to determine the maximum number of barrels of oil per hour that can be sent from 1 to 5.



Quick Summary

- Network flow models are the most frequently solved LPs.
- They are defined on graphs and represent a convenient and flexible modelling tool.
- Network flow models are special cases of LPs and can be solved by applying general purpose methods (e.g., Simplex algorithm).
- If the problem data are integer, basic solutions in Simplex are always integer. Hence, can use Simplex to solve even if flows need to be integer.
- There is a number of special purpose algorithms designed for network flow problems (network Simplex, transportation Simplex, Ford-Fulkerson, Dijkstra's, etc.) which reduce computational time and improve efficiency.